

Run it yourself (Mac)

Everything below runs on a laptop with no outside accounts. Ten minutes the first time, one command after that.

One-time setup

```
# 1. Node 24 (n8n 2.x requires it). If you use nvm:
nvm install 24 && nvm use 24

# 2. Postgres 17 with the pgvector extension (used by the chatbot workflows)
brew install postgresql@17 pgvector

# 3. This repo and its two dev dependencies (n8n itself, and a tiny SMTP sink)
git clone https://github.com/123maria0608-star/n8n-workflows
cd n8n-workflows
npm install          # 3 to 5 minutes; n8n is large
```

Docker is not required. `docker-compose.yml` is here to show the production shape (n8n + Postgres, optional Redis and workers), not for the demo.

Run the whole demo

```
bash demo/demo.sh
```

What you will see, in order:

1. A fresh Postgres cluster in `demo/out/pg` with the `workflow_index` and `chat_log` tables from `demo/schema.sql`.
2. Credentials and the eight workflows imported into a fresh n8n instance in `demo/out/n8n`, then published (activated).
3. Mock Vapi, GoHighLevel and Twilio APIs on port 4010, an SMTP sink on 2525, n8n on 5678. An n8n API key is minted for workflow 08.
4. Ten HTTP events fired at the webhooks, with each response printed.
5. Every mock API call and every email that resulted.
6. The follow-up cron run by hand, the indexer run by hand, three chatbot questions with answers, and the `chat_log` rows in Postgres.

Then it stops everything. Total run time is about a minute.

Look at it in the editor

```
bash demo/serve.sh
```

That brings Postgres, the mocks, the mail sink and n8n back up on the database the demo left behind, and prints two URLs:

- `http://localhost:5678`: the editor. Log in as `demo@example.com` / `Demo-pass-1234`. Open any workflow and click **Executions** to see every run with each node's input and output. Execution #5 is the deliberate failure that triggered the error workflow.
- `http://localhost:5678/webhook/<id>/chat`: the chatbot page. Ask it "what happens when someone texts STOP?" and watch executions 06 and 07 appear.

To run a workflow by hand in the editor, open it and click **Execute workflow**. Workflows 04 and 08 have a Manual Trigger next to their schedule for exactly this.

Ctrl-C in that terminal stops everything.

Fire one event by hand

While `serve.sh` is running, in another terminal:

```
curl -X POST http://localhost:5678/webhook/lead \
  -H 'content-type: application/json' -H 'x-webhook-secret: demo-secret' \
  -d '{"contact_id":"c_104","first_name":"Sam","phone":"5555550104","customData":
{"service":"tint"}}'
```

You get back `{"ok":true,"called":"+15555550104","callId":"call_..."}`. The mock terminal shows the Vapi and GoHighLevel calls. Send it again inside six hours and you get `{"ok":true,"skipped":"duplicate_within_6h"}`.

Point it at real services

Open a workflow, click the **Config** node, and replace the base URLs, secrets and emails with real ones. Then in Settings, Credentials, edit the four credentials with real keys. Nothing else changes. `workflows/*.json` in this repo already carries the production URLs; the demo only swaps them at import time (`demo/prepare-demo.js`).

The one-container quick start (docker run)

```
echo 'ANTHROPIC_API_KEY=sk-ant-...' > .env.local    # only workflow 09 needs it; the file is
git-ignored
bash demo/quickstart-docker.sh
```

One `docker run`, SQLite inside the container, all nine workflows imported and published, then a GET (404 on purpose) and a POST with a support ticket. Read `docs/what-dad-did.md` for what every line means.

The Docker version (what most teams actually run)

```
brew install colima docker docker-compose          # Docker without Docker Desktop
mkdir -p ~/.docker/cli-plugins && ln -sf "$(brew --prefix)/opt/docker-compose/bin/docker-
compose" ~/.docker/cli-plugins/docker-compose
colima start                                       # boots the Linux VM Docker runs in (2-5 min
first time)

pkill -f "n8n start"                             # if the npm demo is still on port 5678
bash demo/docker-demo.sh                          # n8n + Postgres in containers, workflows
imported, one event fired
```

Then open <http://localhost:5678>, click **Executions**, and paste the curl commands the script prints one at a time. Each one shows up in the list within a second. `docker compose ps` shows the two containers, `docker compose logs -f n8n` tails the server, `bash demo/docker-demo.sh down` removes everything.

The difference from the npm demo is only where n8n and Postgres run. The workflow JSON, the credentials file and the mocks are the same; the only values that change are hostnames (`postgres` instead of `localhost:5433`, and `host.docker.internal` so the container can reach the mocks on your laptop).

If something does not start

- `n8n did not start`: read `demo/out/n8n.log`. Usually port 5678 is taken by a previous run; `pkill -f "n8n start"`.
- `Task Broker's port 5679 is already in use`: same thing, an old n8n is still up.

- Postgres errors: `brew info postgresql@17` shows the binary path; set `PGBIN=/that/path/bin` before running the script.
- Everything in `demo/out` is disposable. Delete it and run again.